

Bash Mastery — Assignments

bash-learning curriculum

2026-06-04

Table of Contents

1. Module 1 — Bash Basics
2. Module 2 — Conditionals
3. Module 3 — Loops
4. Module 4 — Functions
5. Module 5 — I/O & Redirection
6. Module 6 — Arrays
7. Module 7 — String Manipulation & Regex
8. Module 8 — Processes, Signals, Traps
9. Module 9 — Text Processing (grep / sed / awk)
10. Module 10 — Error Handling & Debugging
11. Module 11 — Advanced Patterns

Module 1 — Bash Basics

Assignments: Bash Basics

Exercise 1 (Easy): Hello with name from CLI

Problem statement Write a script that accepts one name as the first command-line argument and prints: Hello, NAME!

Expected input/output example

```
./hello_name.sh Hitesh  
Hello, Hitesh!
```

Hint Use \$1 for the first argument.

► Solution

Exercise 2 (Easy): Greet using read

Problem statement Write a script that asks the user for their name, stores it in a variable, and then prints Welcome, NAME!.

Expected input/output example

```
./greet_read.sh  
Enter your name: Sai  
Welcome, Sai!
```

Hint Use read -p to show the prompt and capture input in one line.

► Solution

Exercise 3 (Medium): Sum of two CLI arguments

Problem statement Write a script that takes two numbers from the command line and prints their sum.

Expected input/output example

```
./sum_two.sh 7 5
Sum: 12
```

Hint Use \$1 and \$2, then arithmetic expansion with \$((...)).

► Solution

Exercise 4 (Medium): Print argument count and each argument

Problem statement Write a script that prints: - the script name - the total number of arguments - each argument on its own line

Number the arguments as Arg 1, Arg 2, and so on.

Expected input/output example

```
./show_args.sh apple mango grape
Script: ./show_args.sh
Count: 3
Arg 1: apple
Arg 2: mango
Arg 3: grape
```

Hint Use \$0 for the script name, \$# for the count, and loop through "\$@".

► Solution

Exercise 5 (Medium): Curly braces in variable expansion

Problem statement Create a script with two variables: - prefix="Sai" - name="Hitesh"

Print two lines: - Without braces: ... - With braces: ...

The first line should show what happens when Bash sees \$prefixname. The second line must correctly print SaiHitesh.

Expected input/output example

```
./curly_demo.sh
Without braces:
With braces: SaiHitesh
```

Hint When two variable names touch, Bash can read them as one bigger variable name. Use \${prefix} and \${name} to mark the boundaries clearly.

► Solution

Exercise 6 (Hard): Base name and extension from a filename

Problem statement Write a script that accepts a filename as the first command-line argument and prints: - the original value - the base name without the extension - the extension only

Use Bash parameter expansion only. Do not use external commands such as `basename`, `cut`, or `awk`.

Expected input/output example

```
./file_parts.sh report.txt
File: report.txt
Base name: report
Extension: txt
```

Another example:

```
./file_parts.sh archive.tar.gz
File: archive.tar.gz
Base name: archive.tar
Extension: gz
```

Hint First remove everything before the last `/`, then split the remaining text around the last `..`

► Solution

Module 2 — Conditionals

Assignments: Bash Conditionals

Exercise 1 (Easy · 10 marks): Grade calculator with `if` / `elif` / `else`

Problem statement

Write a script that stores a numeric score in `marks` and prints: - Grade: A for 90 or above - Grade: B for 60 to 89 - Grade: C for anything below 60

Sample I/O

```
./grade.sh
Grade: B
```

Hint

Use one `if`, one `elif`, and one `else`. Compare numbers with `-ge`.

► Solution

Exercise 2 (Easy · 10 marks): Show `[ ]` vs `[[ ]]`

Problem statement

Create a script with `file_name="report.txt"` and print: - Pattern matched if the filename ends in `.txt` - Literal match only when using `[ ]` with a quoted `"*.txt"`

The goal is to show that `[[ $file_name == *.txt ]]` supports pattern matching, while `[ "$file_name" = "*.txt" ]` treats the right side as a normal string.

Sample I/O

```
./pattern_demo.sh
Pattern matched
Literal match only
```

Hint

Use `[[ ... ]]` for glob-style matching and `[ ... ]` for a literal string check.

► Solution

Exercise 3 (Medium · 15 marks): Fix numeric vs string comparison

Problem statement

Write a script with: - a=10 - b=2

Print two lines: - Numeric: a is greater - String: b comes later alphabetically

Use -gt for the numeric check and > inside [[]] for the string check.
This exercise is about understanding -eq, -gt, and =/>.

Sample I/O

```
./compare_values.sh
Numeric: a is greater
String: b comes later alphabetically
```

Hint

10 and 2 behave differently in numeric and string comparisons.

► Solution

Exercise 4 (Medium · 15 marks): File existence checker

Problem statement

Ask the user for a path. Then print: - Regular file exists if it is a file - Directory exists if it is a directory - Path does not exist otherwise

Use file test operators only.

Sample I/O

```
./path_check.sh
Enter a path: Lesson2.sh
Regular file exists
```

Another example:

```
./path_check.sh
Enter a path: modules
Directory exists
```

Hint

Use -f, -d, and -e in the correct order.

► Solution

Exercise 5 (Medium · 20 marks): case with multiple patterns

Problem statement

Write a script that stores a command in action and prints: - Starting service for start, run, or up - Stopping service for stop or down - Unknown action for anything else

Use one case block and combine patterns with |.

Sample I/O

```
./service_case.sh
Starting service
```

Hint

A single branch can match multiple words like start|run|up).

► Solution

Exercise 6 (Hard · 30 marks): Interactive menu with read + case

Problem statement

Build a simple menu-driven script that displays: 1. Show date 2. Show current directory 3. Exit

Read the user's choice and use case to print the correct result. If the input is not 1, 2, or 3, print Invalid choice.

Sample I/O

```
./menu.sh
1) Show date
2) Show current directory
3) Exit
Choose an option: 2
Current directory: /root/projects/bash-learning
```

Hint

Print the menu with echo, capture input with read -r -p, then branch with case.

► Solution

Module 3 — Loops

Module 03 — Loop Assignments

Exercise 1: Print 1..N with a C-style for

Problem Write a Bash script that asks the user for N and prints the numbers from 1 to N using a C-style for loop.

Expected output

```
Enter N: 5
1
2
3
4
5
```

Hint Use for ((i=1; i<=N; i++)).

► Solution

Exercise 2: Sum 1..N with while

Problem Read a number N and calculate the sum of all integers from 1 to N using a while loop.

Expected output

```
Enter N: 5
Sum = 15
```

Hint Keep a counter and a sum variable. Increment the counter each iteration.

► Solution

Exercise 3: Iterate over files and skip hidden ones

Problem Loop over entries in the current directory and print only non-hidden filenames. If a name starts with `.`, skip it with `continue`.

Expected output

```
Visible: Lesson1.sh
Visible: Lesson2.sh
Visible: README.md
```

Hint Use globbing such as `for f in *.*` carefully, or loop over `*` if you only want visible names. To practice `continue`, explicitly test for hidden names.

► Solution

Exercise 4: Break on the first match

Problem Search the current directory for the first file whose name ends with `.sh`. Print the match, then stop the loop immediately.

Expected output

```
First shell script found: Lesson1.sh
```

Hint Use `break` as soon as the condition is true.

► Solution

Exercise 5: Read `/etc/passwd` line by line

Problem Read `/etc/passwd` one line at a time and print usernames whose login shell is **not** `nologin`.

Expected output

```
root
sync
hitesh
```

Hint Use `IFS=:` with `read -r` so each colon-separated field is read safely.

► Solution

Exercise 6: Iterate over an array with proper quoting

Problem Create an array containing values such as "red apple", "green grape", and "banana". Print each element on its own line without breaking words that contain spaces.

Expected output

```
Item: red apple
Item: green grape
Item: banana
```

Hint Use `"${arr[@]}"`, not `${arr[@]}` and not `${...}`.

► Solution

Module 4 — Functions

Module 04: Functions — Assignments

1) Write `is_even()`

Problem: Create a function `is_even()` that accepts one number. It should return success (0) when the number is even and failure (1) when it is odd.

Sample I/O:

```
is_even 8
echo $? # 0
```

```
is_even 5
echo $? # 1
```

Hint: Use arithmetic expansion with `% 2`, and remember that return is for exit status.

► Solution

2) Write `greet(name, greeting='Hello')`

Problem: Write a function `greet()` that prints a greeting. The first parameter is the name, and the second parameter is optional. If the second parameter is missing, use Hello.

Sample I/O:

```
greet "Asha"
## Hello, Asha!
```

```
greet "Asha" "Welcome"
## Welcome, Asha!
```

Hint: Use default values like `${2:-Hello}`.

► Solution

3) Recursive Fibonacci

Problem: Write a recursive function `fib()` that prints the nth Fibonacci number. Use `fib 0 -> 0`, `fib 1 -> 1`, `fib 6 -> 8`.

Sample I/O:

```
fib 0 # 0
fib 1 # 1
fib 6 # 8
```

Hint: You cannot store the result in return for big values. Print the number with `echo` and capture recursive calls with `$(...)`.

► Solution

4) Timestamped `log()`

Problem: Write a function `log()` that prints a message prefixed with a timestamp like `[2025-01-31 09:15:00] Starting app`.

Sample I/O:

```
log "Starting app"
## [2025-01-31 09:15:00] Starting app
```

Hint: Use date `'+%Y-%m-%d %H:%M:%S'` and `"$@"` to print the whole message.

► Solution

5) Validate an IP-looking string

Problem: Write a function `looks_like_ip()` that returns success only if the input looks like `number.number.number.number`. For this exercise, you only need to check the structure, not whether each number is 0-255.

Sample I/O:

```
looks_like_ip "192.168.1.10"
echo $? # 0

looks_like_ip "hello.world"
echo $? # 1
```

Hint: `[[ ... =~ ... ]]` is useful for pattern matching with regular expressions in bash.

► Solution

6) Refactor duplicated code into a function

Problem: Suppose a script repeats this block several times:

```
echo "-----"
echo "Deploying web"
echo "-----"
```

Refactor it into a reusable function `print_section()` that accepts the section name.

Sample I/O:

```
print_section "Deploying web"
## -----
## Deploying web
## -----
```

Hint: If you see repeated code with only one value changing, that changing value should usually become a parameter.

► Solution

Module 5 — I/O & Redirection

Module 05 Assignments: I/O and Redirection

1) Redirect stderr only to a file

Problem: Write a command that runs ls missing-file real-file, keeps normal output on the terminal, and saves only the error message into errors.log.

Sample I/O: - Terminal shows the listing for real-file - errors.log contains the ls: cannot access 'missing-file'... message

Hint: Remember that stderr is file descriptor 2.

► Solution

2) Merge stdout and stderr into one log

Problem: Run a command that prints one normal line and one error line, then store both in combined.log.

Sample I/O: - combined.log ends up with: text hello oops

Hint: Send stdout to the file first, then point stderr to stdout's new location.

► Solution

3) Count lines from stdin via a pipe

Problem: Take a list of values from stdin and count how many lines came in.

Sample I/O:

```
printf 'red\nblue\ngreen\n' | wc -l  
## output: 3
```

Hint: wc -l counts newline-terminated lines from stdin.

► Solution

4) Build a config file with a here-document

Problem: Create a file named app.conf containing:

```
host=localhost  
port=8080  
debug=true
```

Use a here-document instead of multiple echo commands.

Sample I/O: After running your command, cat app.conf should show the three lines above.

Hint: Use cat > app.conf <<EOF and close it with the same marker.

► Solution

5) Compare two find outputs with process substitution

Problem: Compare the file lists of dir_a and dir_b without creating intermediate files. Ignore ordering differences.

Sample I/O:

```
diff <(find dir_a -type f | sort) <(find dir_b -type f | sort)
```

If the directories contain the same file paths, diff prints nothing.

Hint: <(...) makes command output look like a temporary file path.

► Solution

6) Use tee to log while still showing output

Problem: Run a command that prints three status lines, displays them on the terminal, and also stores them in run.log.

Sample I/O: - Terminal shows: text step 1 step 2 done - run.log contains the same three lines

Hint: Put tee at the end of a pipeline.

► Solution

Module 6 — Arrays

Module 06 Assignments: Bash Arrays

1) Reverse an indexed array

Problem

Given an array like nums=(10 20 30 40 50), print the elements in reverse order.

Sample I/O

```
Input array : 10 20 30 40 50
Output      : 50 40 30 20 10
```

Hint

Use indices from \${#nums[@]} and loop backward with for ((i=...; i>=0; i--)).

► Solution

2) Remove duplicates with an associative array

Problem

Given items=(apple banana apple mango banana kiwi), print only the first occurrence of each value.

Sample I/O

```
Input : apple banana apple mango banana kiwi
Output : apple banana mango kiwi
```

Hint

Track seen values with declare -A seen. Only print when a value has not been seen yet.

► Solution

3) Count word frequency from a paragraph

Problem

Take a paragraph, split it into words, and count how many times each word appears.

Sample I/O

```
Paragraph : bash is fun and bash is fast
Output:
bash => 2
is => 2
fun => 1
and => 1
fast => 1
```

Hint

Use read -ra words <<< "\$text" and an associative array like count[word]=\$((count[word]+1)).

► Solution

4) Build a path-like array and join with ':'

Problem

Create an array of directories and print them as a PATH-style string separated by colons.

Sample I/O

```
Input array : /usr/local/bin /usr/bin /bin
Output      : /usr/local/bin:/usr/bin:/bin
```

Hint

Set IFS=: temporarily, then use \${paths[*]} inside double quotes.

► Solution

5) Find the maximum number in an array

Problem

Given scores=(42 17 88 63 88 29), print the maximum numeric value.

Sample I/O

```
Input : 42 17 88 63 88 29
Output : 88
```

Hint

Start with max=\${scores[0]} and compare each value using arithmetic comparison.

► Solution

6) Store env-like key=value pairs in an associative array

Problem

Given lines like `HOST=localhost`, `PORT=8080`, and `MODE=dev`, store them in an associative array and print each key/value pair.

Sample I/O

Input:
`HOST=localhost`
`PORT=8080`
`MODE=dev`

Possible output:
`HOST => localhost`
`PORT => 8080`
`MODE => dev`

Hint

Split each line using parameter expansion: `${line%%=*}` for the key and `${line#*=}` for the value.

► Solution

Module 7 — String Manipulation & Regex

Module 07 Assignments: String Manipulation

1) Extract the extension from a filename

Problem: Given a filename like `photo.jpeg` or `archive.tar.gz`, print only the final extension.

Sample I/O:

Input: `archive.tar.gz`
Output: `gz`

Hint: Use the longest prefix-removal form from the left.

► Solution

2) Slugify a string

Problem: Convert a title into a simple slug: lowercase it, replace spaces with hyphens, and collapse repeated spaces first.

Sample I/O:

Input: `Hello Bash Learners`
Output: `hello-bash-learners`

Hint: Trim or normalize spaces, then use `${var,,}` and `${var// /-}`.

► Solution

3) Validate an email-ish string with =~

Problem: Accept strings that look like name@example.com and reject obvious non-matches.

Sample I/O:

Input: user_42@example.org
Output: valid

Input: not-an-email
Output: invalid

Hint: Use `[[ $value =~ regex ]]` and do **not** quote the regex on the right side.

► Solution

4) Parse name=value lines

Problem: Read lines such as port=8080 and split each line into a key and value.

Sample I/O:

Input: port=8080
Output: key=port value=8080

Hint: Set `IFS='='` for a single read command.

► Solution

5) Mask the middle of a credit-card-like string

Problem: Keep the first 4 and last 4 characters visible, but replace the middle characters with *.

Sample I/O:

Input: 1234567812345678
Output: 1234*****5678

Hint: Use `${#card}` for length and `${card: -4}` for the tail.

► Solution

6) Replace all tabs with 4 spaces

Problem: Convert tab characters in a string into four spaces.

Sample I/O:

Input: name<TAB>score<TAB>grade
Output: name score grade

Hint: A tab can be written as `$(\t)` inside bash.

► Solution

Module 8 — Processes, Signals,

Traps

Module 08 Assignments — Processes, Signals, and Traps

These exercises focus on process control, signals, cleanup, and bounded execution.

Exercise 1: Create a temp directory and clean it with trap

Problem

Write a script that: 1. creates a temporary directory inside the current working directory using `mktemp -d ./scratch.XXXXXX` 2. stores the path in a variable 3. installs `trap 'rm -rf "$temp_dir"' EXIT` 4. creates two files inside that directory 5. prints the directory path while the script is running

Sample I/O

```
$ bash exercise1.sh
Created temp dir: ./scratch.a1B2c3
Wrote files: notes.txt, data.log
Script finished. Cleanup will now run.
```

Hint

Use `temp_dir="$(mktemp -d ./scratch.XXXXXX)"` and define the trap after the variable is assigned.

► Solution

Exercise 2: Run 5 sleeps in parallel and measure total time

Problem

Write a script that launches five `sleep 1` commands in the background, waits for all of them, and prints how many seconds the whole batch took.

Sample I/O

```
$ bash exercise2.sh
Launching 5 jobs in parallel...
All jobs finished.
Total seconds: 1
```

Hint

Capture `start=$(date +%s)` before the loop, then wait, then `end=$(date +%s)`.

► Solution

Exercise 3: Catch Ctrl-C and exit gracefully

Problem

Write a script that loops forever, prints Working..., and sleeps for 1 second each iteration. If the user presses Ctrl-C, the script should print a friendly message and exit with a non-zero status.

Sample I/O

```
$ bash exercise3.sh
Working...
Working...
^CInterrupted by user. Cleaning up and exiting.
```

Hint

Use trap 'echo "Interrupted by user. Cleaning up and exiting."; exit 130' INT.

► Solution

Exercise 4: Spawn a background job and report when it finishes

Problem

Write a script that starts a background command, saves its PID, waits for it, and then prints a message saying the job completed.

Sample I/O

```
$ bash exercise4.sh
Started job with PID 12345
Job 12345 finished successfully.
```

Hint

Start with sleep 2 &, save pid=\$!, then wait "\$pid".

► Solution

Exercise 5: Use timeout to stop a long command

Problem

Write a script that runs a command that would normally take 10 seconds, but use timeout so it is stopped after 2 seconds. Detect the timeout exit code and print a helpful message.

Sample I/O

```
$ bash exercise5.sh
Starting bounded command...
The command timed out after 2 seconds.
```

Hint

timeout usually returns exit code 124 when it kills the command.

► Solution

Exercise 6: Write a retry-with-backoff wrapper

Problem

Write a shell function named `retry_with_backoff` that: 1. accepts a command to run 2. retries up to 3 times 3. waits 1 second before retry 2, then 2 seconds before retry 3 4. stops early if the command succeeds

Test it with a command that fails the first two times and succeeds the third time.

Sample I/O

```
$ bash exercise6.sh
Attempt 1 failed.
Sleeping 1 second before retry...
Attempt 2 failed.
Sleeping 2 seconds before retry...
Attempt 3 succeeded.
```

Hint

Use a loop like `for attempt in 1 2 3; do ... done` and compute sleep time with `sleep "$((attempt))"` for the failed attempts.

► Solution

Module 9 — Text Processing (grep / sed / awk)

Module 09 Assignments: Text Processing

1) Extract email addresses with `grep -oE`

Problem

Given a file `contacts.txt`, print only the email addresses.

Sample input

```
Name: Asha, Email: asha@example.com
Name: Ravi, Email: ravi.kumar@demo.org
Support: helpdesk@company.net
```

Expected output

```
asha@example.com
ravi.kumar@demo.org
helpdesk@company.net
```

Hint

Use `grep -oE` with a regular expression that matches `name@domain.tld`.

► Solution

2) Replace TODO with DONE in all .md files using sed

Problem

Inside the current directory tree, replace every TODO with DONE in Markdown files only.

Sample input

```
notes.md: TODO: add examples
readme.md: TODO: review wording
script.sh: TODO should stay unchanged
```

Expected result

All .md files are updated in place so TODO becomes DONE.

Hint

Use find to locate *.md files, then run sed -i on them. Remember the macOS sed -i " quirk.

► Solution

3) Print users from /etc/passwd whose UID is at least 1000

Problem

Use awk on /etc/passwd and print usernames whose UID is greater than or equal to 1000.

Sample input

```
root:x:0:0:root:/root:/bin/bash
alice:x:1000:1000:Alice:/home/alice:/bin/bash
bob:x:1001:1001:Bob:/home/bob:/bin/zsh
```

Expected output

```
alice
bob
```

Hint

/etc/passwd is colon-separated, so use -F ':'. The UID is field 3 and the username is field 1.

► Solution

4) Build a word-frequency pipeline

Problem

From a paragraph in paragraph.txt, print the most frequent words in descending count order.

Sample input

```
Bash is simple. Bash is fast. Text tools make bash powerful.
```

Sample output

```
3 bash
1 tools
1 text
```

1 simple
1 powerful

Hint

Lowercase the text first, split words onto separate lines, then use `sort | uniq -c | sort -rn`.

► Solution

5) Sum a CSV column with `awk -F','`

Problem

Given this file:

Sample input

```
item,amount
Keyboard,1200
Mouse,450
Monitor,8900
```

Print the sum of the amount column.

Expected output

```
10550
```

Hint

Skip the header row with `NR > 1`.

► Solution

6) Find files larger than 1 MB with `find + xargs ls -la`

Problem

List detailed information for files bigger than 1 MB in the current directory tree.

Sample command goal

```
-rw-r--r-- 1 user user 2456789 Jan 10 12:30 ./logs/big.log
-rw-r--r-- 1 user user 1450011 Jan 10 12:30 ./data/archive.bin
```

Hint

Use `find ... -size +1M -print0` and pair it with `xargs -0` so filenames with spaces are handled safely.

► Solution

Module 10 — Error Handling & Debugging

Module 10 Assignments — Error Handling & Debugging

These exercises reinforce strict mode, defensive scripting, traps, retries, and shellcheck.

1) Refactor a script to use set -euo pipefail safely

Problem

You inherit this script:

```
#!/bin/bash
name=$1
cat "$2" | grep "$name"
echo Found it
```

Refactor it so it safely uses:

```
set -euo pipefail
```

Requirements: - fail with a helpful usage message if fewer than 2 args are passed - avoid the useless cat | grep pipeline - print a friendly “not found” message instead of crashing when grep does not match - keep the script readable

Sample I/O

```
$ ./find-name.sh alice names.txt
Found: alice
```

```
$ ./find-name.sh zoe names.txt
Name 'zoe' was not found.
```

```
$ ./find-name.sh alice
ERROR: usage: ./find-name.sh NAME FILE
```

Hint

set -e is great, but grep returning 1 for “no match” is often normal business logic. Handle that case explicitly with if grep ...; then or if ! grep ...; then.

Solution

► Show solution

2) Write die() and usage() helpers

Problem

Create copy-safe.sh that expects exactly 2 arguments: source and destination.

Requirements: - die() prints ERROR: ... to stderr and exits 1 - usage() prints usage: ./copy-safe.sh SOURCE DEST - reject missing args immediately - reject a missing source file before calling cp

Sample I/O

```
$ ./copy-safe.sh
ERROR: usage: ./copy-safe.sh SOURCE DEST
```

```
$ ./copy-safe.sh missing.txt backup.txt
ERROR: source file not found: missing.txt
```

Hint

Keep helpers tiny. `usage()` can simply call `die`.

Solution

► Show solution

3) Check required commands and dependencies before running

Problem

Write a script named `backup-db.sh` that depends on `tar`, `gzip`, and `date`.

Requirements: - define `require_cmd()` using `command -v` - fail early if any dependency is missing - create an archive name like `backup-2025-01-15.tar.gz` - print a clear success message

Sample I/O

```
$ ./backup-db.sh
All dependencies are available.
Creating backup-2025-01-15.tar.gz
Backup created successfully.
```

Hint

A reusable helper such as `require_cmd tar` is cleaner than repeating the same if block three times.

Solution

► Show solution

4) Add an ERR trap that logs line number and command

Problem

Write a script that: - enables `set -Eeuo pipefail` - installs an `ERR` trap - logs the failing line number and command - intentionally triggers a failure to prove the trap works

Sample I/O

```
$ ./err-demo.sh
Starting demo...
ERROR at line 14 while running: cp missing.txt backup.txt
```

Hint

Use `BASH_COMMAND` for the command text and `LINENO` for the line number. Put the reporting logic in a function to keep the trap readable.

Solution

► Show solution

5) Wrap a flaky command in retry logic with exponential backoff

Problem

Suppose a network command fails intermittently. Write `retry.sh` with a function:

```
retry MAX_ATTEMPTS command args...
```

Requirements: - retry a command up to `MAX_ATTEMPTS` - wait 1, then 2, then 4, then 8 seconds... - stop immediately once the command succeeds - print attempt numbers and delays - fail with a final error after the last attempt

Sample I/O

```
$ ./retry.sh
Attempt 1 failed. Sleeping 1s...
Attempt 2 failed. Sleeping 2s...
Attempt 3 succeeded.
```

Hint

Use a delay variable that starts at 1 and doubles with `delay=$((delay * 2))` after each failed attempt.

Solution

► [Show solution](#)

6) Run shellcheck on a broken script and fix everything

Problem

Start with this intentionally broken script and fix all reported issues:

```
#!/bin/bash
file=$1
if [ -f $file ]; then
    cat $file | grep TODO
    echo Done
fi
for x in $(cat $file); do
    echo $x
done
```

Requirements: - run `shellcheck broken.sh` - fix quoting issues - remove useless `cat` - avoid unsafe `for x in $(cat file)` word splitting - make the script handle missing args safely

Sample I/O

```
$ shellcheck broken.sh
## ... warnings shown ...

$ ./broken.sh tasks.txt
TODO: buy milk
TODO: call plumber
Done
word-safe iteration complete
```

Hint

Expect warnings related to quoting, command substitution, and robustness. Use `while IFS= read -r line; do ... done < "$file"` for safe line-by-line processing.

Solution

► Show solution

Module 11 — Advanced Patterns

Module 11 Assignments — Advanced Bash

These exercises reinforce advanced bash patterns from Lesson11.sh. Each task includes the goal, sample I/O, a hint, and a collapsible solution.

1) Build a short-flag CLI with getopt

Problem

Write `cli-demo.sh` that supports:

- `-h` → print help
- `-v` → enable verbose mode
- `-f FILE` → read a filename

If `-f` is missing, print an error and exit non-zero.

Sample I/O

```
$ ./cli-demo.sh -h
Usage: ./cli-demo.sh [-h] [-v] -f FILE
```

```
$ ./cli-demo.sh -v -f notes.txt
Verbose mode is ON
Processing file: notes.txt
```

```
$ ./cli-demo.sh -v
Error: -f FILE is required
```

Hint

Use `while getopt "hv:f:" opt; do ...` and remember that `f:` means the option needs a value.

Solution

► Show solution

2) Template renderer using here-docs

Problem

Create a script that generates `app.conf` with variable interpolation enabled. Then create `template.txt` where variables are left literal.

Sample I/O

```
$ APP_NAME=demo PORT=9000 ./render.sh
Created app.conf
Created template.txt
```

```
$ cat app.conf
name=demo
port=9000
```

```
$ cat template.txt
name=$APP_NAME
port=$PORT
```

Hint

Use `<<EOF` for interpolation and `<<'EOF'` to disable it.

Solution

► Show solution

3) Integer calculator using (())

Problem

Write `calc.sh` that accepts two integers and prints sum, difference, product, and integer division.

Sample I/O

```
$ ./calc.sh 9 4
sum=13
diff=5
prod=36
div=2
```

Hint

Use arithmetic expansion for printing values and `(( ))` when you want arithmetic statements.

Solution

► Show solution

4) Make a script both sourceable and runnable

Problem

Create `mathlib.sh` with a function `double()` and a `main()` function. When sourced, only the function should load. When executed directly, it should run `main`.

Sample I/O

```
$ source ./mathlib.sh
$ double 7
```

14

```
$ ./mathlib.sh 7  
14
```

Hint

Use the guard `[[ "${BASH_SOURCE[0]}" == "${0}" ]] && main "$@"`.

Solution

► Show solution

5) mkfifo producer/consumer demo

Problem

Use a named pipe called `jobs.pipe`. A producer should send three items into the pipe, and a consumer should print them.

Sample I/O

```
$ ./fifo-demo.sh  
received: task-1  
received: task-2  
received: task-3
```

Hint

Create the FIFO with `mkfifo`, write to it in the background, and read it in a while read loop.

Solution

► Show solution

6) One-liner challenges

Problem

Solve these with compact commands:

1. Print the value of an associative-array key named `theme`.
2. Remove duplicate lines from `names.txt` in place.
3. Replace `localhost` with `127.0.0.1` across all `.conf` files.
4. Print `.port` from `settings.json` using `jq`.

Sample I/O

```
$ declare -A cfg=([theme]=dark)  
$ printf '%s\n' "${cfg[theme]}"  
dark
```

Hint

Think about `${map[key]}`, `awk '!seen[$0]++'`, `grep -rl`, `sed -i`, and `jq`.

Solution

► Show solution